exness_bot — Step-by-Step Guide

LLM + rule-based auto-trading bot for Exness (MetaTrader 5)

Version 1.0 · Author: Waleed Hassan

> **Read this first.** Automated FX/CFD trading loses money for most retail traders. This bot is a **framework and starting point** and has **not** been run against a live account by the author. Always go **backtest** → **demo for weeks** → **tiny live size**, in that order. Algo trading is allowed by Exness; arbitrage, latency exploitation and quote-delay tick-scalping are not. Nothing here is financial advice.

1. What the bot does

Once per **closed candle** (default timeframe M15) it runs this pipeline:

```
MT5 price history
     ▼
indicators   SMA(20), SMA(50), RSI(14), ATR(14), 200-SMA trend
     ▼
strategy     LLM decision (OpenAI)  --falls back to-->  rule set
     ▼                                      (SMA crossover + RSI + trend)
decision     buy / sell / close / hold   (+ confidence 0..1)
     ▼
filters      session window? spread ok? min ATR? confidence >= min?
            daily loss limit not hit? no position already open?
     ▼
risk          lot size from RISK_PER_TRADE_PCT
            SL = SL_ATR_MULT x ATR,  TP = TP_ATR_MULT x ATR
     ▼
executor     mt5.order_send (retries on requote)   [skipped if DRY_RUN]
```

On **every poll** (default every 5 s) it also manages any open position:

- at **+1.0R** profit → move the stop loss to entry (break-even);
- at **+1.5R** profit → trail the stop `2 × ATR` behind price;
- the stop is only ever tightened, never loosened.

R = the initial risk distance (entry → stop loss). All activity is logged to `exness_bot/logs/bot.log`; each entry/exit is appended to `exness_bot/logs/trades.csv`.

2. What every file is for

File	Job
`exness_bot/config.py`	all settings. `DRY_RUN=True`, `DEMO_ONLY=True` by default
`exness_bot/settings.example.py`	copy to `settings.py`; MT5 login + optional OpenAI key (git-ignored)

`exness_bot/mt5_client.py`	connect, account, positions, candles; auto-resolves the broker symbol suffix; reports spread
`exness_bot/indicators.py`	indicator maths + trailing-stop + session logic. No MT5 import, so the backtester reuses it
`exness_bot/data.py`	pulls live candles → DataFrame with indicator columns; drops the still-forming candle
`exness_bot/strategy.py`	builds the prompt, calls the LLM, validates the JSON; rule-based fallback
`exness_bot/risk.py`	lot sizing, SL/TP, break-even + trailing stop, daily-loss guard, session check
`exness_bot/executor.py`	`mt5.order_send` for open / close / modify-stop, with retries; obeys `DRY_RUN`
`exness_bot/runner.py`	the main loop that ties it together
`exness_bot/backtest.py`	runs the rule strategy over history and prints edge metrics

3. Every setting in `config.py`

Safety

Setting	Default	Meaning
`DRY_RUN`	`True`	`True` = never send a real order, only log what it would do
`DEMO_ONLY`	`True`	refuse to start if the connected account is a live account
`CONFIRM_LIVE_STRING`	`""`	must equal `"I ACCEPT THE RISK"` to run on a live account

Market

Setting	Default	Meaning
`SYMBOL`	`EURUSD`	base name; broker suffix (`m`, `.`, `z` …) found automatically
`TIMEFRAME`	`M15`	candle used for decisions
`LOOKBACK_BARS`	`400`	candles pulled for indicators / LLM context

Risk

Setting	Default	Meaning
`RISK_PER_TRADE_PCT`	`0.5`	percent of balance lost if the stop is hit
`FIXED_LOT_FALLBACK`	`0.01`	lot used if sizing can't be computed
`MAX_LOT`	`0.10`	hard cap on lot size
`MAX_OPEN_POSITIONS`	`1`	positions allowed at once on this symbol
`ATR_PERIOD`	`14`	ATR length
`SL_ATR_MULT`	`2.0`	stop distance = this × ATR
`TP_ATR_MULT`	`3.0`	target distance = this × ATR
`MIN_ATR_POINTS`	`0`	skip entries when ATR (points) is below this; `0` = off

`DAILY_MAX_LOSS_PCT`	`3.0`	stop opening trades for the day after this equity drawdown
`MIN_CONFIDENCE`	`0.55`	ignore buy/sell signals weaker than this

Trailing / break-even

Setting	Default	Meaning
`BREAKEVEN_AT_R`	`1.0`	move stop to entry once profit reaches this many R; `0` = off
`TRAIL_AT_R`	`1.5`	start trailing once profit reaches this many R; `0` = off
`TRAIL_ATR_MULT`	`2.0`	trailing stop sits this × ATR behind price

Entry filters

Setting	Default	Meaning
`MAX_SPREAD_POINTS`	`25`	skip entries when the spread is wider than this
`USE_TREND_FILTER`	`True`	only long in an up-trend, only short in a down-trend
`TREND_SMA`	`200`	SMA used as the trend proxy
`TREND_SLOPE_LOOKBACK`	`20`	bars back used to measure the trend SMA slope
`SESSION_UTC_HOURS`	`[7, 16]`	trade only between these UTC hours; `[]` = always
`TRADE_DAYS`	`[0,1,2,3,4]`	weekdays allowed (Mon–Fri)

Execution

Setting	Default	Meaning
`DEVIATION_POINTS`	`20`	maximum slippage accepted, in points
`MAGIC`	`20240601`	id stamped on this bot's orders
`POLL_SECONDS`	`5`	how often the loop checks for a new candle / manages the stop

`ORDER_RETRIES`	`3`	retries on requote / price-changed
`TRADE_LOG_CSV`	`exness_bot/logs/trades.csv`	where trades are recorded

Strategy

Setting	Default	Meaning
`USE_LLM`	`True`	use the LLM if an API key is set, else the rule set
`SMA_FAST` / `SMA_SLOW`	`20` / `50`	crossover SMAs
`RSI_PERIOD`	`14`	RSI length

Backtest

Setting	Default	Meaning
`BT_SPREAD_POINTS`	`12`	spread cost (points) charged on every backtested trade
`BT_COMMISSION_PER_LOT`	`7.0`	informational commission assumption for Raw/Zero accounts
`BT_START_BALANCE`	`1000`	starting balance for the money curve

4. Step-by-step: backtest (do this first)

Runs on **any OS** — no MetaTrader, no Windows needed.

1. Install Python 3.10+ and pandas:

   ```
   pip install pandas
   ```

2. Get historical candles as a CSV with columns `time, open, high, low, close` (export from MT5: *View* → *Symbols* → *Bars*, or any data provider). Save as e.g. `data/EURUSD_M15.csv`.
3. Run:

   ```
   python -m exness_bot.backtest --csv data/EURUSD_M15.csv
   ```

   On Windows with MT5 connected you can pull data directly instead:

   ```
   python -m exness_bot.backtest --mt5 180
   ```

4. Read the report:

   ```
   trades         : 15
   win rate       : 26.7%
   expectancy     : -0.604R per trade
   profit factor  : 0.30
   total          : -9.1R
   max drawdown   : 4.5%
   verdict        : NO EDGE - do not trade this as-is
   ```

   - **expectancy > 0** and **profit factor > 1.15** → maybe an edge, forward-test it.
   - otherwise → change the strategy or parameters and run again.
5. Per-trade detail is written to `exness_bot/logs/backtest_trades.csv`.

> Never move to live money on a “NO EDGE” result.

5. Step-by-step: demo on Windows

The `MetaTrader5` Python package only runs on **Windows** (a Windows VPS is fine).

1. Create a **demo** account in the Exness Personal Area; note the login number, password and server (e.g. `Exness-MT5Trial`).
2. Install the **Exness MetaTrader 5** terminal, log into the demo account.
3. In the terminal: **Tools** → **Options** → **Expert Advisors** → **Allow algorithmic trading** (tick it).
4. Install dependencies:

   ```
   pip install -r requirements.txt
   ```

5. Create your private config:

   ```
   copy exness_bot\settings.example.py exness_bot\settings.py
   ```

   Edit `exness_bot\settings.py`:

   ```
   MT5_PATH     = r"C:\Program Files\MetaTrader 5 EXNESS\terminal64.exe"
   MT5_LOGIN    = 12345678
   MT5_PASSWORD = "your-demo-password"
   MT5_SERVER   = "Exness-MT5Trial"
   OPENAI_API_KEY = ""          # leave empty to use the rule-based strategy
   OPENAI_MODEL   = "gpt-4o-mini"
   ```

   (`settings.py` is git-ignored — credentials never get committed.)

   First run with `DRY_RUN = True` (the default):

   ```
   python -m exness_bot.runner
   ```

6. Watch `exness_bot/logs/bot.log`. You'll see decisions and lines like `[DRY_RUN] would OPEN buy 0.02 EURUSD ...`. No orders are sent.
7. When the dry-run behaviour looks sane, set `DRY_RUN = False` in `exness_bot/config.py` and run again. Now it trades the **demo** account for real. Let it run for **several weeks**; review `logs/trades.csv`.

6. Step-by-step: going live (only after profitable demo)

In `exness_bot/config.py`:

```
DEMO_ONLY = False
CONFIRM_LIVE_STRING = "I ACCEPT THE RISK"
MAX_LOT = 0.01
RISK_PER_TRADE_PCT = 0.25      # start smaller than on demo
```

2. Put your **live** account details in `settings.py`.
3. Run `python -m exness_bot.runner`. It logs a warning that it is on a live account.
4. Watch it daily for the first weeks. Keep the daily loss limit tight.
5. Scale size up only after months of consistent results, never faster than your drawdown tolerance.

Keep the machine/VPS on and the MT5 terminal running. If the terminal closes, the bot loses its connection — open positions keep their broker-side SL/TP, but the trailing logic pauses until it reconnects.

7. How good is it for Exness — honest assessment

a) Will it work *technically* on Exness?

Fairly likely after a short debugging pass on demo. Usual first-run snags:

Snag	Fix
wrong `MT5_SERVER` string	`initialize failed` → copy the exact server name from the terminal
symbol is `EURUSDm` / `EURUSD.z` on your account	handled automatically; check the log line `Resolved symbol EURUSD -> ...`
"AutoTrading disabled"	step 5.3 above
broker min stop distance > your ATR stop	the bot clamps it; on very tight timeframes SL/TP get pushed out — use a higher timeframe

Rough chance of a clean demo run within a day or two: **~85%**.

b) Will it make real profit?

The built-in strategy (SMA crossover + RSI + trend filter, or a generic LLM prompt) is a **baseline, not an edge**. After spread, swap and commission its expected value is around zero-to-negative. Realistic odds that this exact configuration is net-profitable over a year: **~10–15%**, in line with retail algo-trading outcomes generally.

What actually moves that number:

- A strategy with a tested statistical edge — validate with `backtest.py` over several years and multiple symbols, then walk-forward.
- Trading the right sessions / instruments for that edge.
- Costs: a low-spread account type; avoid holding across the 3-day swap.
- Risk discipline: small `RISK_PER_TRADE_PCT`, a hard daily stop — which this bot already enforces.

> **Bottom line:** treat this as a solid *execution and risk* framework with a *backtester* attached. The profitable idea has to be yours and has to survive the backtest before any money is at stake.

8. Troubleshooting

Symptom	Cause / fix
`settings.py not found`	you didn't copy `settings.example.py` → `settings.py`
`mt5.initialize failed`	wrong `MT5_PATH`, `MT5_SERVER`, login or password; terminal not installed
`Symbol EURUSD(+suffix) not found`	your account uses a different name; add its suffix to `_SUFFIXES` in `mt5_client.py`
Decisions logged but no orders	`DRY_RUN` is still `True`
`OPEN rejected: retcode=10027`	AutoTrading disabled in the terminal
`OPEN rejected: retcode=10016`	invalid stops (broker min distance); raise `SL_ATR_MULT` or use a higher timeframe
`OPEN rejected: retcode=10019`	not enough money for that lot; lower `RISK_PER_TRADE_PCT` or `MAX_LOT`
No trades ever	trend filter + session window too strict, or `MIN_CONFIDENCE` too high; loosen and re-backtest
LLM errors in the log	bad/empty `OPENAI_API_KEY`; the bot auto-falls back to the rule set

9. Safe-launch checklist

- [ ] Backtested on ≥ 2 years of data, on ≥ 2 symbols → positive expectancy, PF > 1.15
- [ ] Walk-forward / out-of-sample check also positive
- [ ] Ran on demo with `DRY_RUN=True`, decisions look sane in the log
- [ ] Ran on demo with `DRY_RUN=False` for ≥ 4 weeks → result matches the backtest
- [ ] `RISK_PER_TRADE_PCT` ≤ 0.5, `DAILY_MAX_LOSS_PCT` ≤ 3, `MAX_LOT` small
- [ ] Account type has low spread; you understand swap on your instrument
- [ ] VPS / machine stays on; MT5 terminal set to start with the OS
- [ ] You can afford to lose the whole account balance

Only when every box is ticked: set the two live flags and start at `0.01` lots.